LAPORAN PRAKTIKUM
ANALISIS KOMPLEKSITAS ALGORITMA

OLEH:

Nama	: RachmadPamuji
Nim	: 2025573010103
Kelas	: TI 1\C
Jurusan	: Teknologi Informasi dan Komputer
Dosen	: Muhammad Reza Zulman.S.ST.,M.Sc.

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2026

LEMBAR PENGESAHAN

Nama Mata Kuliah	: Praktikum Algoritma Dan Struktur Data
Judul Pratikum	: Praktikum Analisis Kompleksitas Algoritma
Hari/Tanggal Pratikum	: 21 April 2026
Tanggal Penyerahan	: 28 April 2026
Tempat	: Laboratorium (TIK.106)
Nama Mahasiswa	: Rachmad Pamuji
NIM	: 2025573010103
Program Studi/Kelas	: Teknik Informatika / TI.1C

Mengetahui
Dosen Pengajar,

Muhammad Reza Zulman.S.ST.,M.Sc.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum yang berjudul *"Praktikum Analisis Kompleksitas Algoritma"* dengan baik dan tepat waktu.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk penyempurnaan laporan ini di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah membantu dalam penyusunan laporan ini, baik secara langsung maupun tidak langsung.

Semoga laporan ini dapat bermanfaat bagi pembaca dan memberikan gambaran yang jelas mengenai bahasa pemrograman java script.

DAFTAR ISI

LEMBAR PENGESAHAN i

KATA PENGANTAR ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Pengertian Dan Cara Kerja Dari Big O Nation 1

BAB II PROSEDUR KERJA 3

2.1 Membuat Folder Dan File Package.json 3

2.2 Intro Kompleksitas 3

2.3 Notasi Big-O 4

2.4 Space Complexity 7

2.5 Refactor 9

BAB III PENUTUP 11

3.1 Kesimpulan 11

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam dunia komputasi modern, algoritma merupakan fondasi utama untuk menyelesaikan berbagai masalah secara efisien. algoritma yang baik tidak hanya memberikan solusi yang benar, tetapi juga harus optimal dalam penggunaan sumber daya seperti waktu eksekusi dan ruang memori. Analisis kompleksitas algoritma menjadi alat krusial untuk mengukur efisiensi tersebut, terutama melalui notasi Big O yang menggambarkan perilaku terburuk (worst-case), rata-rata (average-case), dan terbaik (best-case) dari suatu algoritma terhadap ukuran input (n).pentingnya analisis ini semakin terasa seiring pertumbuhan data besar (big data) dan aplikasi real-time, seperti pencarian di mesin pencari atau pemrosesan transaksi di e-commerce. **t**anpa analisis kompleksitas, pengembang berisiko memilih algoritma yang lambat, seperti pengurutan bubble sort dengan kompleksitas (O(n^2)), padahal alternatif seperti quicksort (O(n \log n) jauh lebih efisien. di indonesia, perkembangan teknologi informasi mendorong pemahaman mendalam tentang topik ini, sebagaimana tercermin dalam kurikulum pendidikan komputer di berbagai perguruan tinggi.

Laporan ini bertujuan menganalisis kompleksitas berbagai algoritma umum, seperti pencarian linear dan biner, serta pengurutan dasar, untuk memberikan pemahaman praktis bagi pemula dalam pemrograman JavaScript atau bahasa serupa. Dengan demikian, diharapkan pembaca dapat menerapkan prinsip ini dalam pengembangan aplikasi web yang skalabel.

1.2 Tujuan Praktikum

Praktikum ini bertujuan untuk memahami konsep dasar,dan perintah perintah dasar yang ada di bahasa pemrograman java script.

- Memahami konsep dasar kompleksitas algoritma menggunakan notasi Big O.
- Membandingkan hasil pengukuran untuk menentukan efisiensi relatif antar algoritma.
- Mampu untuk menganalisis kompleksitas loop tunggal dan nested loop.

1.3 Pengertian Dan Cara Kerja Dari Big O Nation

Big O Notation adalah notasi matematis yang digunakan untuk menggambarkan kompleksitas algoritma dalam hal waktu eksekusi dan penggunaan memori terhadap ukuran input n. Secara sederhana, Big O memberi tahu kita "seberapa buruk" algoritma bisa berjalan saat data bertambah besar, dengan fokus pada worst-case scenario (kasus terburuk).Cara kerjanya adalah dengan menghitung jumlah operasi dasar (seperti perbandingan if, assignment =, atau increment i++) yang dilakukan algoritma,

lalu mengabaikan konstanta dan istilah rendah. Misalnya, jika algoritma melakukan $3n^2 + 2n + 100$ operasi, maka kompleksitasnya adalah $O(n^2)$ karena istilah n^2 tumbuh paling cepat.Analisis dimulai dengan mengidentifikasi pola loop dalam kode. Loop tunggal for(i=0; i<n; i++) menghasilkan $O(n)$, sedangkan nested loop (loop dalam loop) seperti bubble sort menghasilkan $O(n^2)$. Big O juga mengklasifikasikan berdasarkan tiga kasus: best-case (input ideal), average-case (input acak), dan worst-case (input terburuk), tapi praktikum biasanya fokus pada worst-case.

Contoh konkret dalam JavaScript: fungsi akses array arr[0] adalah $O(1)$ karena konstan, sedangkan linear search yang memeriksa setiap elemen hingga ketemu target adalah $O(n)$. Pengukuran empiris dilakukan dengan performance.now() untuk membandingkan waktu eksekusi antar algoritma pada dataset berbeda ukuran

.

BAB II
PROSEDUR KERJA

2.1 Membuat Folder Dan File Package.json

Untuk membuat folder dan file package.json yang harus kita lakukan adalah dengan membuat folder utama terlebih dahulu Ex.(Praktikum-4) kemdian buka folder tersebut ke visual studio code .selanjutnya buka git bash dan ketik perintah npm init -y maka file package.json akan otomatis terbuat.jika file package.json sudah ada selanjutnya adalah membuat file untuk codingan java script, Ex: nama_file.js

2.2 Intro Kompleksitas

Buat file intro-komplesitas dengan cara buka terminal lalu ketik “touch intro-kompleksitas” jika sudah buka file tersebut.

Codingan kompleksitas algoritma :

```
function ukurWaktu(label, fn){
  const mulai =  Date.now();
  fn ();
  const selasai = Date.now();
  console.log(`${label} : ${selasai - mulai} ms`);

}
const N = 100_100 ;

function JumlahkanLinear (n){
  let total = 0 ;
  for ( let i = 1 ;i<=n ; i++) total +=i ;
  return total ;
}
function JumlahkanRumus(n){
  return ( n * ( n + 1)) / 2 ;
}

function Caripasangan (arr){
  const pasangan = []
```

```
  for (let i = 0 ;  i<arr.length ; i ++){
    for (let j=1 +1 ; j < arr.length ; j ++){
      if (arr[i] + arr [j]=== 10) pasangan.push([arr[i],arr[i]]);
    }
  }
  return pasangan;
}

const data = Array.from ({length : 5000 }, (_, i) => i);

console.log('===Perbandingan Waktu Eksekusi===');
ukurWaktu('0(1) JumlahkanRumus',()=> JumlahkanRumus(N));
ukurWaktu('0(n) JumlahkanLinear', ()=> JumlahkanLinear(N));
ukurWaktu('0(n^2) Caripasangan', () => Caripasangan(data));

console.log('\nhasil sama?',JumlahkanLinear(100) === JumlahkanRumus(100));
```

Jalankan dengan perintah node <nama_file>.
Output :

```
C:\Users\agung\2025573010103_Praktikum_Struktur_Data\praktikum-5>node 01-intro-kompleksitas.js
===Perbandingan Waktu Eksekusi===
0(1) JumlahkanRumus : 0 ms
0(n) JumlahkanLinear : 3 ms
0(n^2) Caripasangan : 81 ms

hasil sama? true

C:\Users\agung\2025573010103_Praktikum_Struktur_Data\praktikum-5>
```

Penjelasan : Program di atas digunakan untuk membandingkan waktu eksekusi beberapa algoritma dengan kompleksitas berbeda, yaitu O(1), O(n), dan O(n^2). pengukuran dilakukan dengan fungsi ukurwaktu yang menghitung selisih waktu sebelum dan sesudah suatu fungsi dijalankan. fungsi jumlahkanrumus memiliki kompleksitas O(1) karena menggunakan rumus, sedangkan jumlahkanlinear memiliki kompleksitas O(n) karena menggunakan perulangan. sementara itu, fungsi caripasangan memiliki kompleksitas O(n^2) karena menggunakan nested loop untuk mencari pasangan angka dalam array. Hasilnya menunjukkan bahwa semakin besar kompleksitas algoritma, maka waktu eksekusinya juga semakin lama, terutama pada data berukuran besar.

2.3 Notasi Big-O

Buat file dengan cara buka terminal lalu ketik “touch big-o” kemudian buka file tersebut.

Codingan Big-O :

```
function ambilPertama(arr) { return arr [0];}
function tambahkanItem(arr ,item ){ arr.push(item); }
function isGenap(n) { return n % 2 === 0 ;}
```

```
function binarySearch (arr , target ) {
  let kiri = 0, kanan = arr.length -1 ;
  let langkah = 0 ;
  while(kiri <= kanan) {
    langkah++ ;
    const tengah = Math.floor((kiri + kanan) / 2);
    if ( arr [tengah] === target){
      console.log(`Ditemukan Di index ${tengah} setelah ${langkah} langkah`);
      return tengah ;
    }
    if (arr[tengah] < target) kiri = tengah + 1 ;
    else kanan=kanan -1 ;
  }
  return -1 ;
}

function cariMax (arr){
  let maks = arr [0];
  for (const x of arr ) if ( x > maks ) maks = x;
  return maks ;
}
function bubbleSortNaif(arr){
  const a = [ ...arr] ;
  for (let i = 0 ; i < a.length ; i ++)
    for ( let j = 0 ; j < a.length -i-1 ;j++)
  if (a[j] > a[j+1]) [ a[j]] , a[j+1] = [a[j+1], a[j]];
  return a ;
}
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2);
}

console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42)); // true
console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452);
console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);
  console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
  for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
console.log("");
```

```
console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");
console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
  t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');
```

Jalankan dengan perintah node <nama_file>.

Output :

```
C:\Users\agung\2025573010103_Praktikum_Struktur_Data\praktikum-5>node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
Ditemukan Di index 731452 setelah 37096 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n2) — Bubble Sort (kecil saja) ===
[ 64, [ 34, 64 ], 25, [ 12, 25 ], 22, [ 11, 22 ], 90 ]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
205 ms
Waktu fib(35) O(n) memoization:
0 ms
```

Penjelasan : Big O merupakan cara untuk mengelompokkan algoritma berdasarkan tingkat efisiensinya, dimulai dari yang paling cepat hingga paling lambat. kompleksitas O(1) bersifat konstan, artinya waktu eksekusi tidak dipengaruhi oleh ukuran input, seperti pada akses elemen array atau operasi aritmatika. selanjutnya O(log n) bersifat logaritmik, di mana setiap langkah mengurangi masalah menjadi setengah, contohnya pada binary search. kompleksitas O(n) disebut linear karena waktu eksekusi bertambah sebanding dengan jumlah data, seperti pada perulangan sederhana atau pencarian linear. di atasnya ada O(n log n) yang sering digunakan pada algoritma sorting efisien seperti merge sort dan quick sort. kemudian $O(n^2)$ merupakan kompleksitas kuadratik yang biasanya muncul pada nested loop dan cenderung lambat untuk data besar. yang paling lambat adalah $O(2^n)$, yaitu eksponensial, di mana waktu eksekusi meningkat sangat cepat dan hanya cocok untuk input kecil, seperti pada perhitungan fibonacci rekursif.

Dalam penyederhanaannya, Big O hanya memperhatikan pertumbuhan terbesar. konstanta diabaikan, sehingga O(2n) disederhanakan menjadi O(n), dan O(100) menjadi O(1). selain itu, jika terdapat beberapa suku, hanya suku dengan pertumbuhan paling dominan yang diperhatikan, misalnya $O(n^2 + n)$ menjadi $O(n^2)$, dan O(n + log n) menjadi O(n). dengan demikian, big O membantu memahami seberapa efisien suatu algoritma ketika ukuran data semakin besar.

2.4 Space Complexity

Buat file dengan cara buka terminal lalu ketik perintah “touch space-complexity” kemudian buka file tersebut .

Codingan Space Complexity :

```
function jumlahArray(arr) {
  let total = 0;
  for (const x of arr) total += x;
  return total;
}
function duplikasiArray(arr) {
  const baru = [];
  for (const x of arr) baru.push(x * 2);
  return baru;
}
function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1);
}
function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i;
  return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log("Jumlah array :", jumlahArray(arr));
console.log("Duplikasi array:", duplikasiArray(arr));
```

```
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
function hitungUnik(arr) {
  const seen = new Set();
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak));
```

Jalankan dengan menggunakan perintah node di terminal Ex.(node spaceComplexity).

Output:

```
C:\Users\agung\2025573010103_Praktikum_Struktur_Data\praktikum-5>node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 0

C:\Users\agung\2025573010103_Praktikum_Struktur_Data\praktikum-5>
Ln 36, Col 1   Spaces: 2   UTF-8
```

Penjelasan : Space Complexity adalah ukuran jumlah memori tambahan (auxiliary space) yang digunakan oleh suatu algoritma seiring bertambahnya ukuran input, tanpa menghitung data input itu sendiri. konsep ini penting selain waktu eksekusi, karena sebuah algoritma tidak hanya harus cepat tetapi juga efisien dalam penggunaan memori. algoritma yang hanya memakai beberapa variabel sederhana termasuk dalam O(1) karena penggunaan memorinya tetap. jika algoritma membuat struktur data baru seperti array berukuran n, maka kompleksitas ruangnya menjadi O(n). begitu juga pada rekursi, jika pemanggilan fungsi terjadi hingga n tingkat, maka akan membutuhkan ruang O(n) pada call stack.sering terjadinya trade-off antara waktu dan memori (time-space trade-off), di mana algoritma bisa dibuat lebih cepat dengan menggunakan lebih banyak memori, atau sebaliknya. contohnya pada teknik memoization dalam perhitungan fibonacci yang dapat mengubah waktu eksekusi dari $O(2^n)$ menjadi O(n), tetapi membutuhkan tambahan memori sebesar O(n).

Contoh Nyata	Pola Kode	Kompleksitas
arr[0], obj.key, Map.get()	Akses array/object langsung	O(1)

forEach, for loop linear	Satu loop melewati n elemen	O(n)
Binary Search, halving loop	Loop i *= 2 atau i /= 2	O(log n)
2n → O(n) setelah buang konstanta	Dua loop berurutan (bukan nested)	O(n)
Bubble Sort, pencarian pasangan sederhana	Dua loop bersarang (nested)	$O(n^2)$
Algoritma pada operasi matrix	Tiga loop bersarang	$O(n^3)$
Fibonacci naif, power set	Rekursi bercabang dua	$O(2^n)$
Merge Sort, Quick Sort (rata-rata)	Rekursi + sorting	O(n log n)

2.5 Refactor

Buat file dengan cara buka terminal lalu ketik “touch refactor.js” kemudia buka file tersebut
Codingan refactor :

```
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
  return false;
}

function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}
function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x);
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}
function frekuensiCepat(arr) {
  const counter = {};
```

```
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}

const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);
let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");
const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

Jalankan dengan menggunakan perintah node refactor.js di terminal.

Output :

```
C:\Users\agung\2025573010103_Praktikum_Struktur_Data\praktikum-5>node 04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }

C:\Users\agung\2025573010103_Praktikum_Struktur_Data\praktikum-5>
Ln 45, Col 1 (1201 selected)   Spaces: 2   UTF-8
```

Penjelasan : Codingan refactor digunakan untuk membandingkan efisiensi algoritma dalam mendeteksi duplikat dan menghitung frekuensi data. fungsi ada duplikatlambat menggunakan nested loop sehingga memiliki kompleksitas $O(n^2)$ dan cenderung lambat, sedangkan adaDuplikatCepat memanfaatkan Set sehingga hanya membutuhkan satu loop dengan kompleksitas O(n) dan lebih cepat. Pada perhitungan frekuensi, frekuensiLambat menggunakan indexOf yang membuatnya kurang efisien, sementara frekuensiCepat memakai objek sebagai counter sehingga lebih optimal dengan kompleksitas O(n). di bagian akhir, program menguji waktu eksekusi menggunakan data besar untuk menunjukkan bahwa algoritma dengan kompleksitas lebih kecil akan berjalan lebih cepat.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum di atas dapat di simpulkan bahwa ,analisis kompleksitas algoritma digunakan untuk mengukur efisiensi suatu program dari segi waktu (time complexity) dan memori (space complexity). dengan menggunakan notasi Big O, kita dapat mengetahui bagaimana performa algoritma ketika ukuran input semakin besar, mulai dari yang paling efisien seperti O(1), hingga yang paling lambat seperti $O(n^2)$ dan $O(2^n)$. hasil percobaan menunjukkan bahwa algoritma dengan kompleksitas lebih rendah cenderung memiliki waktu eksekusi yang lebih cepat, terutama pada data berukuran besar.

Selain itu, pemilihan struktur data dan metode yang tepat sangat berpengaruh terhadap performa program, seperti penggunaan set atau objek yang mampu mengurangi kompleksitas dari $O(n^2)$ menjadi O(n). Di sisi lain, perlu juga mempertimbangkan penggunaan memori melalui konsep space complexity, karena sering terjadi trade-off antara kecepatan dan penggunaan ruang. Oleh karena itu, pemahaman terhadap kompleksitas algoritma sangat penting agar dapat merancang program yang efisien, optimal, dan sesuai dengan kebutuhan.